

Exceptional Control Flow: Signals

Introduction to Computer Systems
8th Lecture, Nov. 10, 2025

ECF Exists at All Levels of a System

■ Exceptions

- Hardware and operating system kernel software

■ Process Context Switch

- Hardware timer and kernel software

■ Signals

- Kernel software and application software

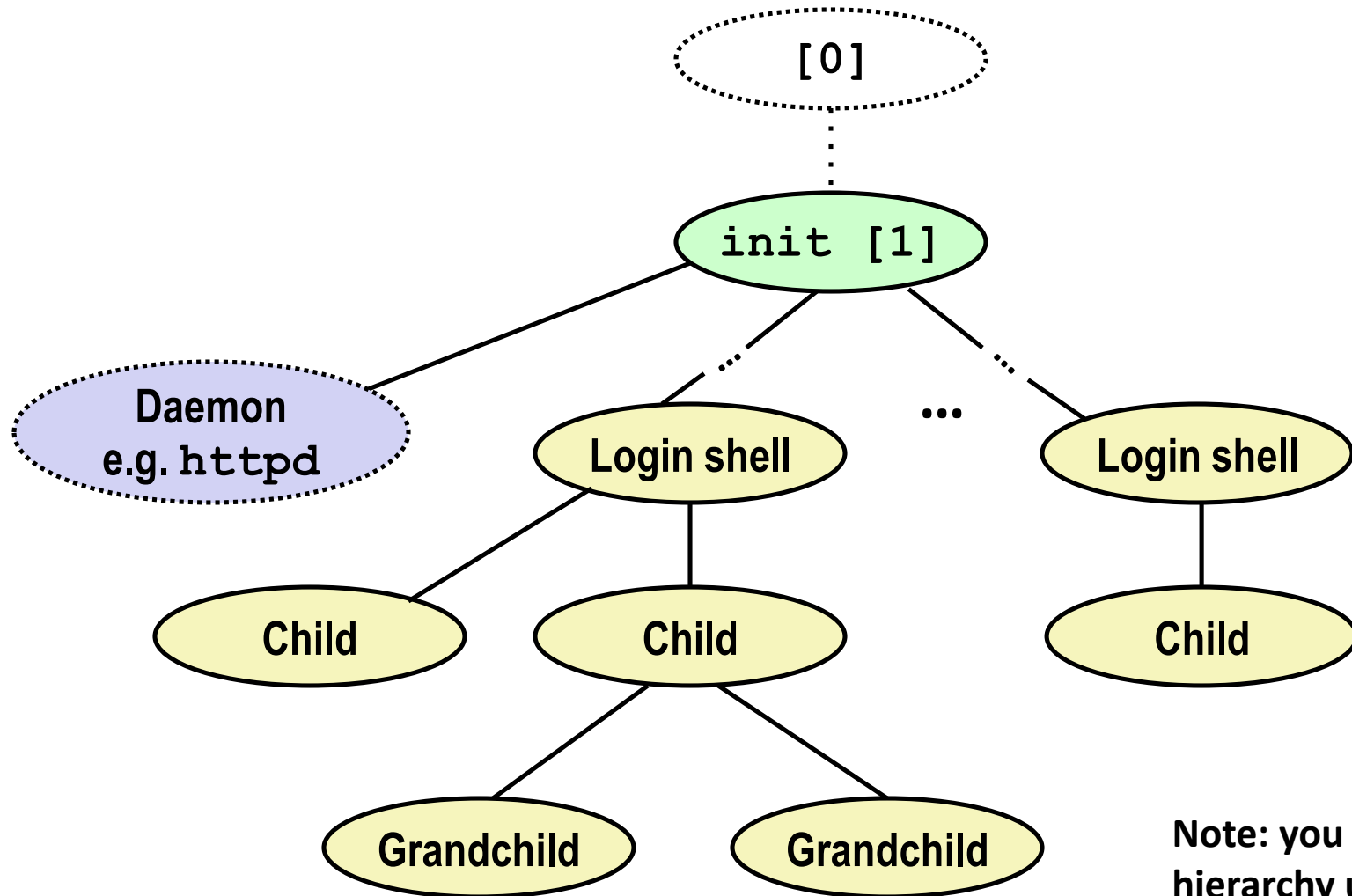
} **Previous Lecture**

} **This Lecture**

Today

- Shells
- Signals

Linux Process Hierarchy



Note: you can view the hierarchy using the Linux `ps tree` command

Shell Programs

- A *shell* is an application program that runs programs on behalf of the user.

- `sh` Original Unix shell (Stephen Bourne, AT&T Bell Labs, 1977)
- `csch/tcsch` BSD Unix C shell
- `bash` “Bourne-Again” Shell (default Linux shell)

```
int main()
{
    char cmdline[MAXLINE]; /* command line */

    while (1) {
        /* read */
        printf("> ");
        Fgets(cmdline, MAXLINE, stdin);
        if (feof(stdin))
            exit(0);

        /* evaluate */
        eval(cmdline);
    }
}
```

shelllex.c

*Execution is a
sequence of
read/evaluate
steps*

Simple Shell eval Function

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE]; /* Holds modified command line */
    int bg; /* Should the job run in bg or fg? */
    pid_t pid; /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

shellex.c

Problem with Simple Shell Example

- Our example shell correctly waits for and reaps foreground jobs
- But what about background jobs?
 - Will become zombies when they terminate
 - Will never be reaped because shell (typically) will not terminate
 - Will create a memory leak that could run the kernel out of memory

ECF to the Rescue!

■ Solution: Exceptional control flow

- The kernel will interrupt regular processing to alert us when a background process completes
- In Unix, the alert mechanism is called a *signal*

Today

- Shells
- Signals

Signals

- A ***signal*** is a small message that notifies a process that an event of some type has occurred in the system
 - Akin to exceptions and interrupts
 - Sent from the kernel (sometimes at the request of another process) to a process
 - Signal type is identified by small integer ID's (1-30)
 - Only information in a signal is its ID and the fact that it arrived

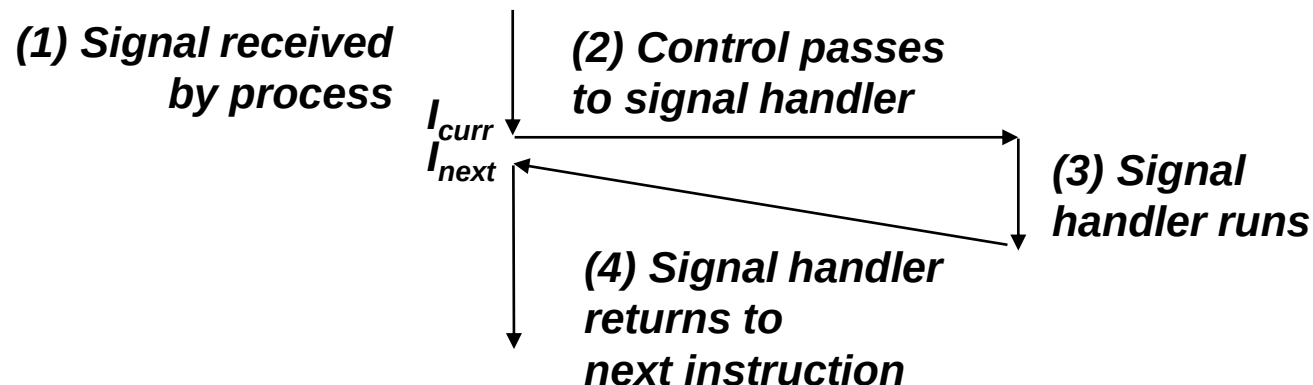
<i>ID</i>	<i>Name</i>	<i>Default Action</i>	<i>Corresponding Event</i>
2	SIGINT	Terminate	User typed ctrl-c
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

Signal Concepts: Sending a Signal

- Kernel *sends* (delivers) a signal to a *destination process* by updating some state in the context of the destination process
- Kernel sends a signal for one of the following reasons:
 - Kernel has detected a system event such as divide-by-zero (SIGFPE) or the termination of a child process (SIGCHLD)
 - Another process has invoked the `kill` system call to explicitly request the kernel to send a signal to the destination process

Signal Concepts: Receiving a Signal

- A destination process *receives* a signal when it is forced by the kernel to react in some way to the delivery of the signal
- Some possible ways to react:
 - *Ignore* the signal (do nothing)
 - *Terminate* the process (with optional core dump)
 - *Catch* the signal by executing a user-level function called *signal handler*
 - Akin to a hardware exception handler being called in response to an asynchronous interrupt:



Signal Concepts: Pending and Blocked Signals

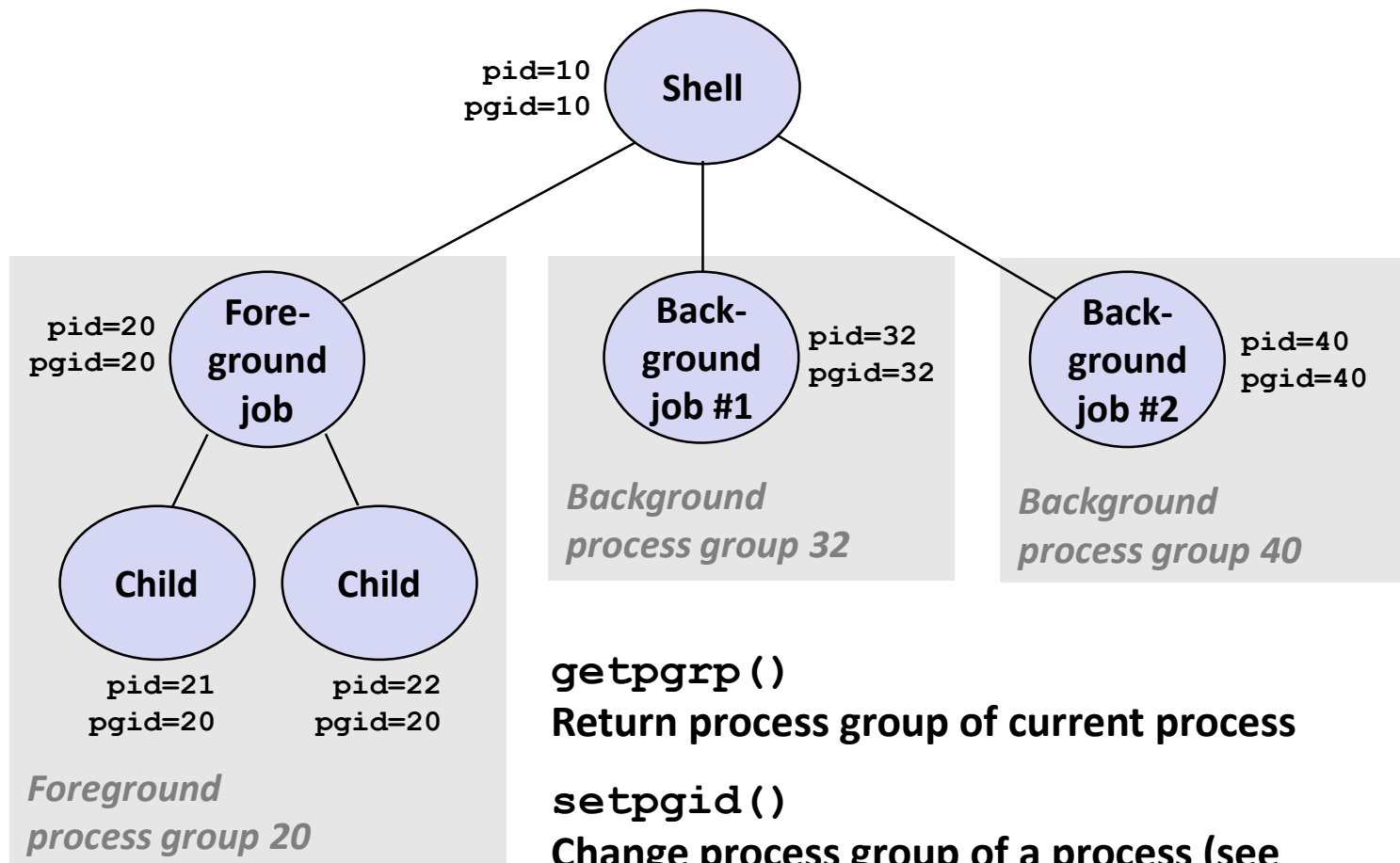
- A signal is *pending* if sent but not yet received
 - There can be at most one pending signal of any particular type
 - Important: Signals are not queued
 - If a process has a pending signal of type k, then subsequent signals of type k that are sent to that process are discarded
- A process can *block* the receipt of certain signals
 - Blocked signals can be delivered, but will not be received until the signal is unblocked
- A pending signal is received at most once

Signal Concepts: Pending/Blocked Bits

- Kernel maintains **pending** and **blocked** bit vectors in the context of each process
 - **pending**: represents the set of pending signals
 - Kernel sets bit k in **pending** when a signal of type k is delivered
 - Kernel clears bit k in **pending** when a signal of type k is received
 - **blocked**: represents the set of blocked signals
 - Can be set and cleared by using the **sigprocmask** function
 - Also referred to as the *signal mask*.

Sending Signals: Process Groups

- Every process belongs to exactly one process group



Sending Signals with `/bin/kill` Program

- `/bin/kill` program
sends arbitrary signal to a
process or process group

- Examples

- `/bin/kill -9 24818`
Send SIGKILL to process 24818

- `/bin/kill -9 -24817`
Send SIGKILL to every process
in process group 24817

```
linux> ./forks 16
Child1: pid=24818 pgrp=24817
Child2: pid=24819 pgrp=24817
```

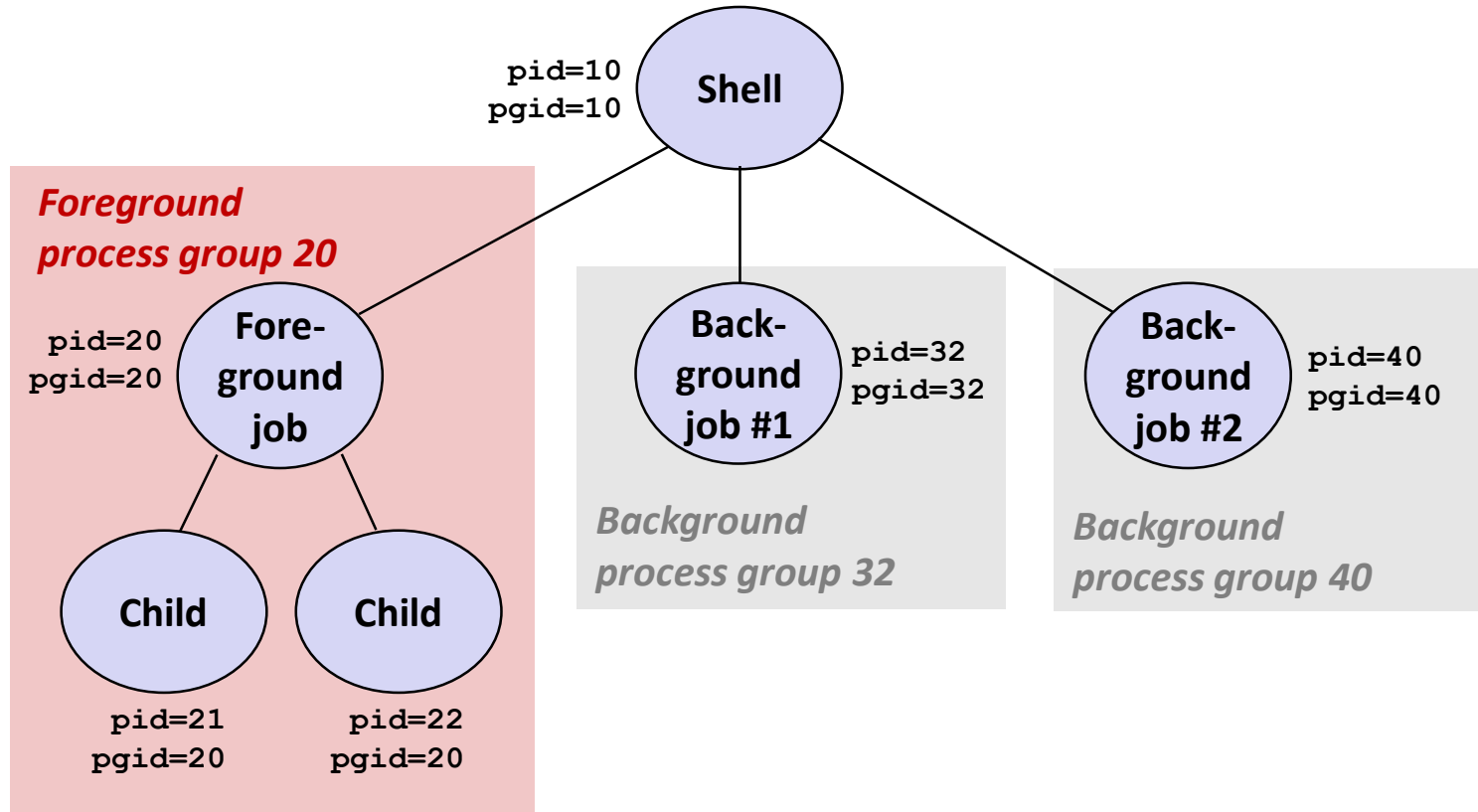
```
linux> ps
  PID TTY          TIME CMD
24788 pts/2        00:00:00 tcsh
24818 pts/2        00:00:02 forks
24819 pts/2        00:00:02 forks
24820 pts/2        00:00:00 ps
```

```
linux> /bin/kill -9 -24817
```

```
linux> ps
  PID TTY          TIME CMD
24788 pts/2        00:00:00 tcsh
24823 pts/2        00:00:00 ps
linux>
```

Sending Signals from the Keyboard

- Typing ctrl-c (ctrl-z) causes the kernel to send a SIGINT (SIGTSTP) to every job in the foreground process group.
 - SIGINT – default action is to terminate each process
 - SIGTSTP – default action is to stop (suspend) each process



Example of `ctrl-c` and `ctrl-z`

```
bluefish> ./forks 17
Child: pid=28108 pgrp=28107
Parent: pid=28107 pgrp=28107
<types ctrl-z>
Suspended
bluefish> ps w
```

PID	TTY	STAT	TIME	COMMAND
27699	pts/8	Ss	0:00	-tcsh
28107	pts/8	T	0:01	./forks 17
28108	pts/8	T	0:01	./forks 17
28109	pts/8	R+	0:00	ps w

```
bluefish> fg
./forks 17
<types ctrl-c>
bluefish> ps w
```

PID	TTY	STAT	TIME	COMMAND
27699	pts/8	Ss	0:00	-tcsh
28110	pts/8	R+	0:00	ps w

STAT (process state) Legend:

First letter:

S: sleeping

T: stopped

R: running

Second letter:

s: session leader

+: foreground proc group

See “man ps” for more details

Sending Signals with kill Function

```
void fork12()
{
    pid_t pid[N];
    int i;
    int child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            /* Child: Infinite Loop */
            while(1)
                ;
        }

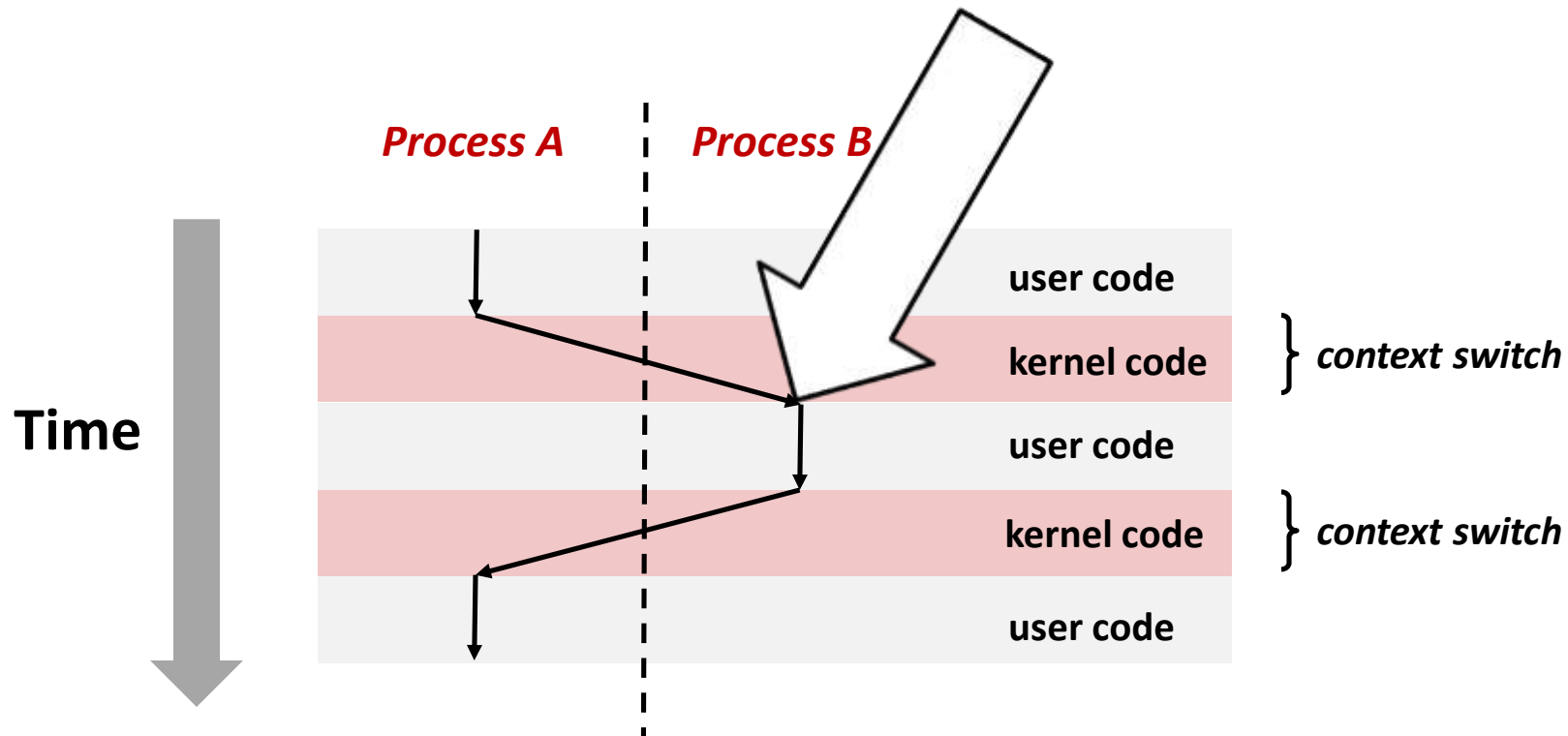
    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```

forks.c

Receiving Signals

- Suppose kernel is returning from an exception handler and is ready to pass control to process p



Receiving Signals

- Suppose kernel is returning from an exception handler and is ready to pass control to process p
- Kernel computes $\text{pnb} = \text{pending} \ \& \ \sim\text{blocked}$
 - The set of pending nonblocked signals for process p
- If $(\text{pnb} == 0)$
 - Pass control to next instruction in the logical flow for p
- Else
 - Choose least nonzero bit k in pnb and force process p to *receive* signal k
 - The receipt of the signal triggers some *action* by p
 - Repeat for all nonzero k in pnb
 - Pass control to next instruction in logical flow for p

Default Actions

- Each signal type has a predefined *default action*, which is one of:
 - The process terminates
 - The process stops until restarted by a SIGCONT signal
 - The process ignores the signal

Installing Signal Handlers

- The `signal` function modifies the default action associated with the receipt of signal `signum`:
 - `handler_t *signal(int signum, handler_t *handler)`
- Different values for `handler`:
 - `SIG_IGN`: ignore signals of type `signum`
 - `SIG_DFL`: revert to the default action on receipt of signals of type `signum`
 - Otherwise, `handler` is the address of a user-level *signal handler*
 - Called when process receives signal of type `signum`
 - Referred to as *“installing”* the handler
 - Executing handler is called *“catching”* or *“handling”* the signal
 - When the handler executes its return statement, control passes back to instruction in the control flow of the process that was interrupted by receipt of the signal

Signal Handling Example

```
void sigint_handler(int sig) /* SIGINT handler */
{
    printf("So you think you can stop the bomb with ctrl-c, do you?\n");
    sleep(2);
    printf("Well...");
    fflush(stdout);
    sleep(1);
    printf("OK. :-)\n");
    exit(0);
}

int main()
{
    /* Install the SIGINT handler */
    if (signal(SIGINT, sigint_handler) == SIG_ERR)
        unix_error("signal error");

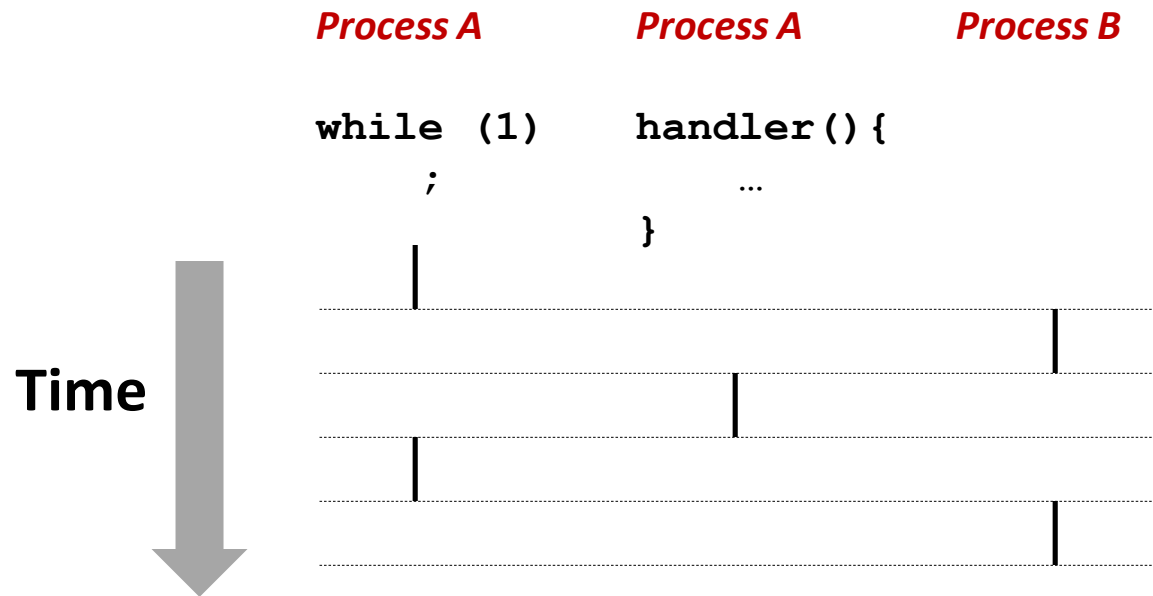
    /* Wait for the receipt of a signal */
    pause();

    return 0;
}
```

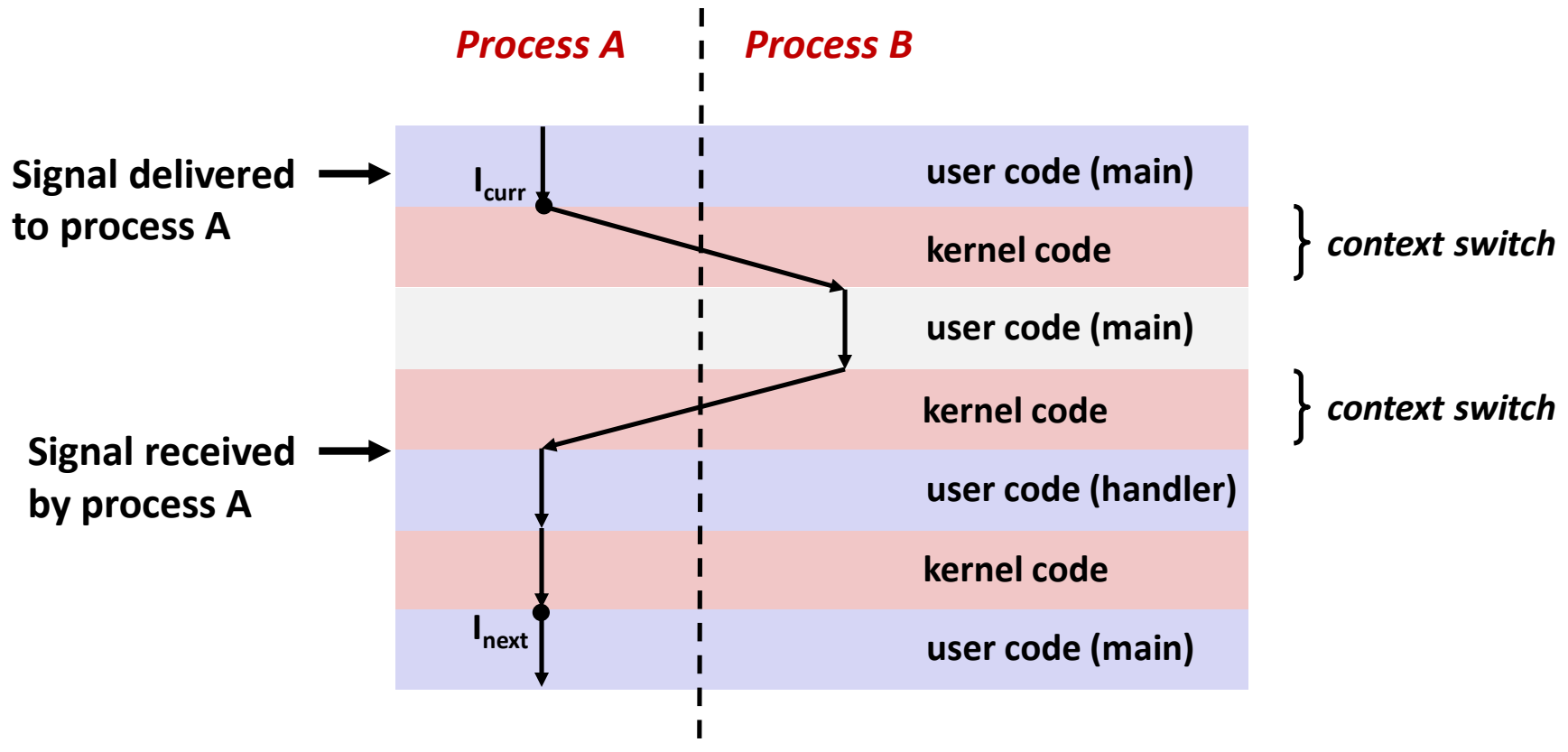
sigint.c

Signals Handlers as Concurrent Flows

- A signal handler is a separate logical flow (not process) that runs concurrently with the main program

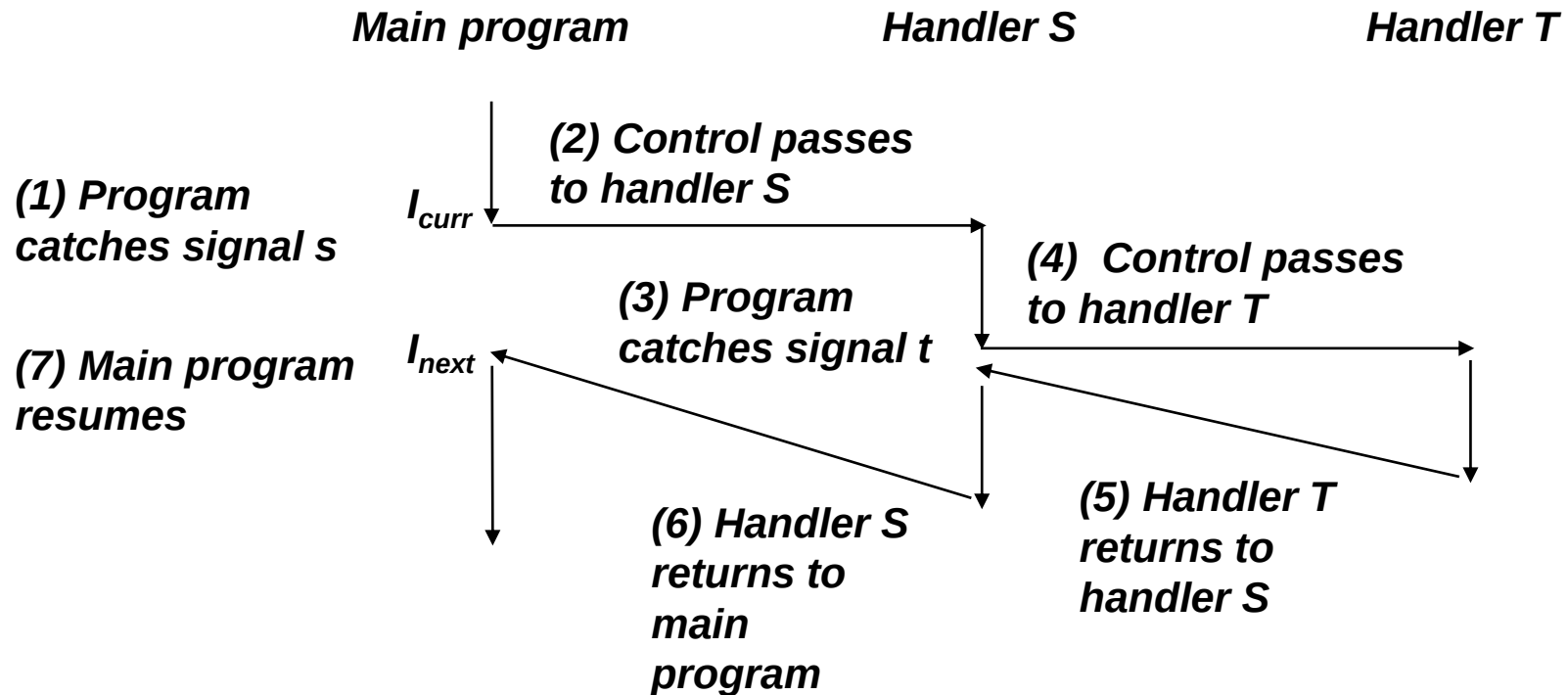


Another View of Signal Handlers as Concurrent Flows



Nested Signal Handlers

- Handlers can be interrupted by other handlers



Blocking and Unblocking Signals

■ Implicit blocking mechanism

- Kernel blocks any pending signals of type currently being handled.
- E.g., A SIGINT handler can't be interrupted by another SIGINT

■ Explicit blocking and unblocking mechanism

- `sigprocmask` function

■ Supporting functions

- `sigemptyset` – Create empty set
- `sigfillset` – Add every signal number to set
- `sigaddset` – Add signal number to set
- `sigdelset` – Delete signal number from set

Temporarily Blocking Signals

```
sigset_t mask, prev_mask;
```

```
Sigemptyset(&mask);  
Sigaddset(&mask, SIGINT);
```

```
/* Block SIGINT and save previous blocked set */  
Sigprocmask(SIG_BLOCK, &mask, &prev_mask);
```

```
⋮  
/* Code region that will not be interrupted by SIGINT */
```

```
/* Restore previous blocked set, unblocking SIGINT */  
Sigprocmask(SIG_SETMASK, &prev_mask, NULL);
```

Safe Signal Handling

- **Handlers are tricky because they are concurrent with main program and share the same global data structures.**
 - Shared data structures can become corrupted.
- **We'll explore concurrency issues later in the term.**
- **For now here are some guidelines to help you avoid trouble.**

Guidelines for Writing Safe Handlers

- **G0: Keep your handlers as simple as possible**
 - e.g., Set a global flag and return
- **G1: Call only async-signal-safe functions in your handlers**
 - `printf`, `sprintf`, `malloc`, and `exit` are not safe!
- **G2: Save and restore `errno` on entry and exit**
 - So that other handlers don't overwrite your value of `errno`
- **G3: Protect accesses to shared data structures by temporarily blocking all signals.**
 - To prevent possible corruption
- **G4: Declare global variables as `volatile`**
 - To prevent compiler from storing them in a register
- **G5: Declare global flags as `volatile sig_atomic_t`**
 - *flag*: variable that is only read or written (e.g. `flag = 1`, not `flag++`)
 - Flag declared this way does not need to be protected like other globals

Summary

- **Signals provide process-level exception handling**
 - Can generate from user programs
 - Can define effect by declaring signal handler
 - Be very careful when writing signal handlers